2.4.5 Going beyond one-dimensional objective functions

So far, we have only seen examples of GPs trained on one-dimensional objective functions. However, there is nothing inherent about a GP that confines us to just one dimension. In fact, as long as our mean and covariance function can handle high-dimensional inputs, a GP can operate in high dimensions without difficulty. In this subsection, we will learn how to train a GP on a two-dimensional data set.

We will follow the procedure in the previous section. First, we need a training data set. Here, I'm artificially creating a dummy set with points at (0, 0), (1, 2), and (-1, 1) with respective labels of 0, -1, 0.5. In other words, the objective function we're learning from has a value of 0 at (0, 0), a value of -1 at (1, 2), and 0.5 at (-1, 1). We'd like to make predictions within the [-3, 3]-by-[-3, 3] square.

This is set up in Python as follows:

```
# training data
train_x = torch.tensor(
    [
        [0., 0.],
        [1., 2.],
        [-1., 1.]
    ]
)

train_y = torch.tensor([0., -1., 0.5])

# test data
grid_x = torch.linspace(-3, 3, 101)   ❶

grid_x1, grid_x2 = torch.meshgrid(grid_x, grid_x, indexing="ij")   ❷
xs = torch.vstack([grid_x1.flatten(), grid_x2.flatten()]).transpose(-1, -2)
```

❶ One-dimensional grid.

❷ Two-dimensional grid.

The variable `xs` is a 10,000-by-2 matrix that contains the latices of a grid over the square we'd like to predict on. (There are 10,000 points because we are taking a 100-segment grid in each of the two dimensions.) Now, we simply rerun the GP code we have before to train a GP and make predictions on this two-dimensional data set. Note that no modification to our `BaseGPModel` class or any of the prediction code is needed, which is quite amazing!

One thing we do need to change though is how we visualize our predictions. As we are operating in two dimensions, plotting the predictive mean and CI in one single plot becomes more difficult. Here, a typical solution is to draw a heat map for the predictive mean, and another heat map for the predictive standard deviation. While the standard deviation is not exactly the 95% CI, these two objects in essence do quantify the same thing: our uncertainty about the function values.

So, instead of calling `predictive_distribution.confidence_region()` as we did before,

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>